

MODUL DESIGN PATTERN 2

FACTORY & SINGLETON



Disusun Oleh : Dionisio Raditya

Pemrograman Berorientasi Obyek, Informatika,
Universitas Atma Jaya Yogyakarta

Modul 11

Design Pattern 2

Design pattern merupakan solusi umum yang dapat digunakan Kembali (*reusable solution*) untuk menyelesaikan permasalahan tertentu dalam pengembangan perangkat lunak. Dalam konsep Object Oriented Programming (OOP), design pattern menjadi salah satu best practice dalam penulisan kode program agar sistem lebih terstruktur, mudah dipahami, serta lebih mudah untuk dikembangkan dan dikelola.

Dengan menerapkan design pattern, pengembang dapat menyusun kode program yang lebih fleksibel, efisien, dan memiliki tingkat *maintainability* yang lebih baik.

Karakteristik Design Pattern:

- Reusability
Pola (pattern) yang telah ada dapat digunakan kembali untuk menyelesaikan permasalahan serupa pada berbagai kasus pengembangan perangkat lunak.
- Standarization
Penggunaan design pattern membantu menciptakan standardisasi dalam penulisan kode program sehingga mempermudah proses pemahaman, komunikasi, dan kerja sama antar programmer.
- Efficiency
Design pattern menyediakan solusi yang telah teruji sehingga proses pengembangan perangkat lunak dapat dilakukan lebih cepat dan efisien.
- Flexibility
Design pattern bersifat abstrak dan tidak bergantung pada implementasi tertentu sehingga dapat diterapkan pada berbagai kebutuhan dan skenario sistem.

Jenis-jenis Design Pattern

Secara umum, design pattern dibagi menjadi tiga kategori utama, yaitu:

- Creational Design Pattern
- Structural Design Pattern
- Behavioral Design Pattern

Pada modul 11 – Design Pattern 2, pembahasan akan difokuskan pada Creational Design Pattern, khususnya pada penerapan Factory Method dan Singleton Pattern

Creational Design Pattern

Creational Design Pattern merupakan kategori design pattern yang berfokus pada proses pembuatan objek (*object creation*). Pattern ini bertujuan untuk membuat sistem lebih fleksibel dan independent dalam menciptakan serta mengelola objek.

Dengan menggunakan Creational Design Pattern, proses instansiasi objek dapat dilakukan secara lebih terstruktur sehingga mengurangi ketergantungan langsung terhadap implementasi objek tertentu.

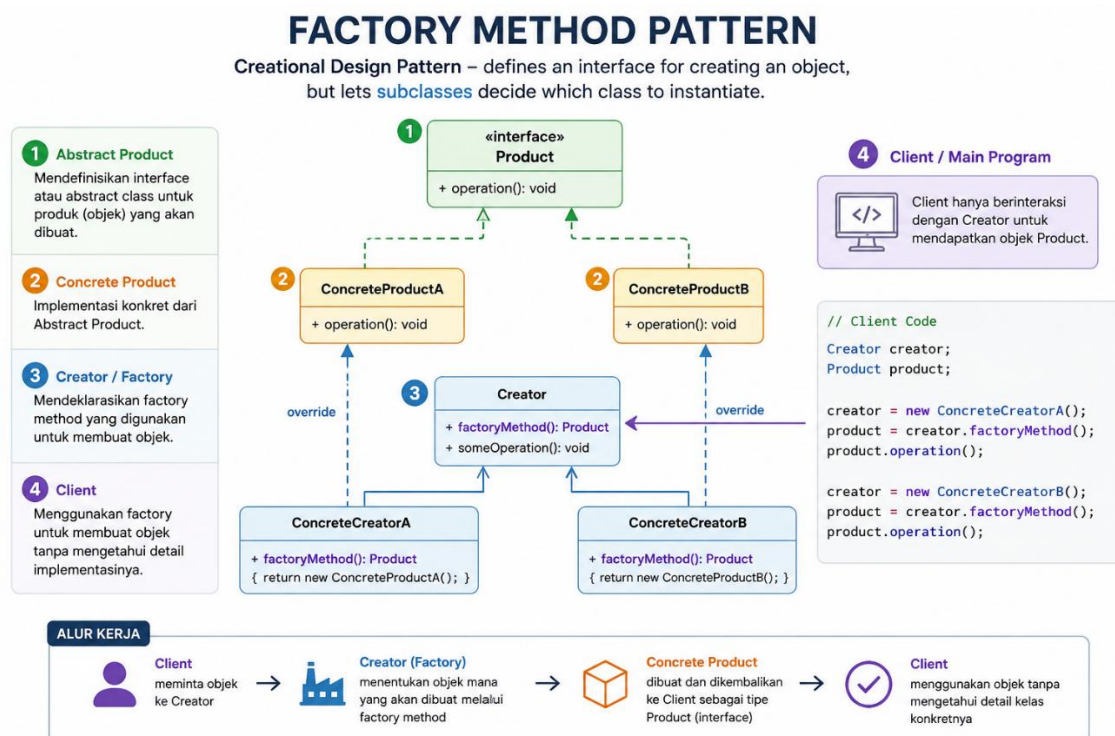
Terdapat lima jenis utama dalam Creational Design Pattern, yaitu:

- Factory Method
- Abstract Factory
- Singleton
- Prototype
- Builder

A. Factory Method

Factory Method merupakan salah satu jenis Creational Design Pattern yang menyediakan antarmuka (interface) atau kelas abstrak (abstract class) untuk proses pembuatan objek. Pattern ini memungkinkan subclass menentukan jenis objek yang akan dibuat tanpa harus mengekspos proses pembuatan objek tersebut kepada client.

Dengan menggunakan **Factory Method**, proses instansiasi objek menjadi lebih terstruktur dan fleksibel karena pembuatan objek dipusatkan pada sebuah method khusus (factory method).

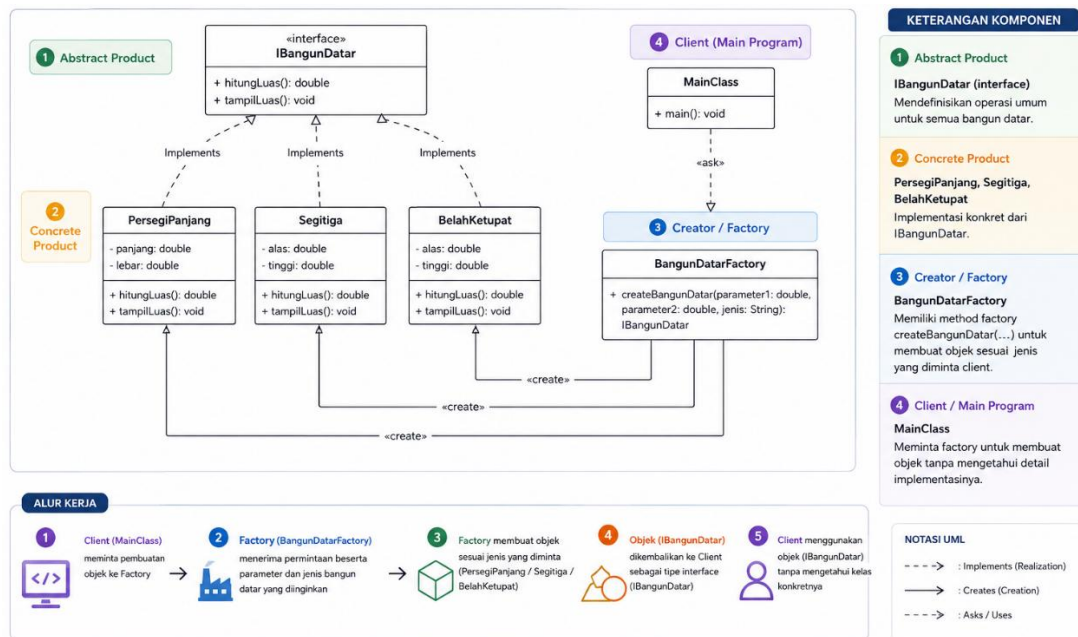


Manfaat Factory Method

- **Menyederhanakan kode pada sisi client**
Client tidak perlu mengetahui detail proses pembuatan objek sehingga kode program menjadi lebih sederhana, terutama ketika terdapat banyak variasi objek dan kondisi tertentu.
- **Mengurangi dependensi antar kelas**
Proses pembuatan objek disembunyikan melalui abstract class atau interface sehingga perubahan implementasi objek tidak terlalu memengaruhi kode pada sisi client.
- **Mempermudah pengelolaan variasi objek**
Factory Method mempermudah pengelolaan berbagai jenis objek karena seluruh proses pembuatan objek dilakukan secara terpusat pada factory.
- **Meningkatkan fleksibilitas sistem**
Penambahan variasi objek baru dapat dilakukan dengan lebih mudah tanpa harus mengubah banyak kode yang sudah ada.

Komponen Factory Method

- **Abstract Product**
Abstract Product merupakan interface atau abstract class yang mendefinisikan perilaku umum dari seluruh objek yang dapat dibuat oleh factory.
- **Concrete Product**
Concrete Product merupakan implementasi nyata dari Abstract Product. Kelas ini mengimplementasikan (implements) interface atau mewarisi (extends) abstract class dari Abstract Product.
- **Creator/ Factory**
Creator atau Factory merupakan kelas yang bertanggung jawab untuk membuat objek sesuai dengan permintaan client. Kelas ini biasanya memiliki method khusus (factory method) untuk menentukan jenis objek yang akan dibuat.
- **Client/ Main Program**
Client merupakan program utama yang menggunakan Creator / Factory untuk meminta pembuatan objek tanpa perlu mengetahui detail implementasi objek tersebut.



B. Singleton Method

Singleton Pattern merupakan salah satu jenis Creational Design Pattern yang memastikan sebuah kelas hanya memiliki satu objek (single instance) selama program berjalan. Pattern ini juga menyediakan titik akses global (global access point) untuk mengakses objek tersebut.

Pada Singleton Pattern, konstruktor dibuat dengan **visibilitas private** agar objek tidak dapat dibuat secara langsung dari luar kelas. Sebagai gantinya, akses terhadap objek dilakukan melalui sebuah method khusus yang akan mengembalikan instance tunggal dari kelas tersebut.

Singleton Pattern umumnya digunakan pada kasus di mana hanya dibutuhkan satu objek bersama, seperti koneksi database, konfigurasi aplikasi, logger, atau manajemen resource sistem.

Prinsip Singleton Pattern

- **Single Instance**
Singleton memastikan bahwa sebuah kelas hanya memiliki satu objek (instance) selama siklus hidup aplikasi.
- **Global Access**
Singleton menyediakan titik akses global sehingga objek dapat diakses dari berbagai bagian program tanpa perlu membuat objek baru.
- **Controlled Instantiation**
Proses pembuatan objek dikontrol sepenuhnya oleh kelas itu sendiri sehingga objek tidak dapat dibuat secara sembarangan dari luar kelas.
- **Thread Safety**

Pada implementasi tertentu, Singleton dapat dirancang agar aman digunakan pada lingkungan multithreading sehingga mencegah terbentuknya lebih dari satu instance secara bersamaan.

- **Private Constructor**

Konstruktor dibuat dengan visibilitas private untuk membatasi proses instansiasi objek secara langsung dari luar kelas.

Komponen Singleton Pattern

- **Static Member**

Singleton memiliki atribut static yang digunakan untuk menyimpan satu-satunya instance dari kelas tersebut.

- **Private Constructor**

Konstruktor dibuat private agar objek tidak dapat dibuat menggunakan keyword new dari luar kelas.

- **Static Factory Method**

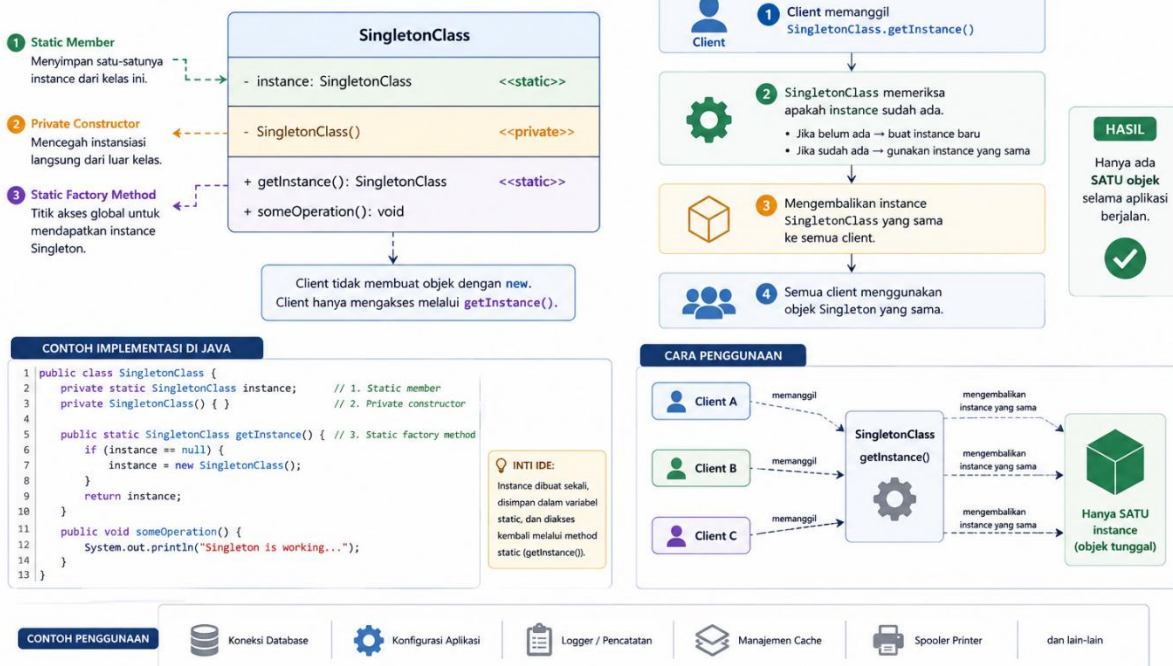
Singleton menyediakan method static, biasanya bernama getInstance(), yang berfungsi sebagai titik akses global untuk mendapatkan instance Singleton.

Method ini akan:

- Membuat objek apabila instance belum tersedia, atau
- Mengembalikan objek yang sudah ada apabila instance telah dibuat sebelumnya.

SINGLETON PATTERN

Memastikan sebuah kelas hanya memiliki satu objek (instance) dan menyediakan titik akses global ke objek tersebut.



Cara kerja Singleton Pattern

1. Client memanggil method `getInstance()`.
2. Kelas Singleton memeriksa apakah instance sudah pernah dibuat.
3. Jika belum ada, kelas akan membuat instance baru.
4. Jika instance sudah ada, method akan mengembalikan instance yang sama.
5. Seluruh client akan menggunakan objek Singleton yang sama.

Kelebihan Singleton Pattern

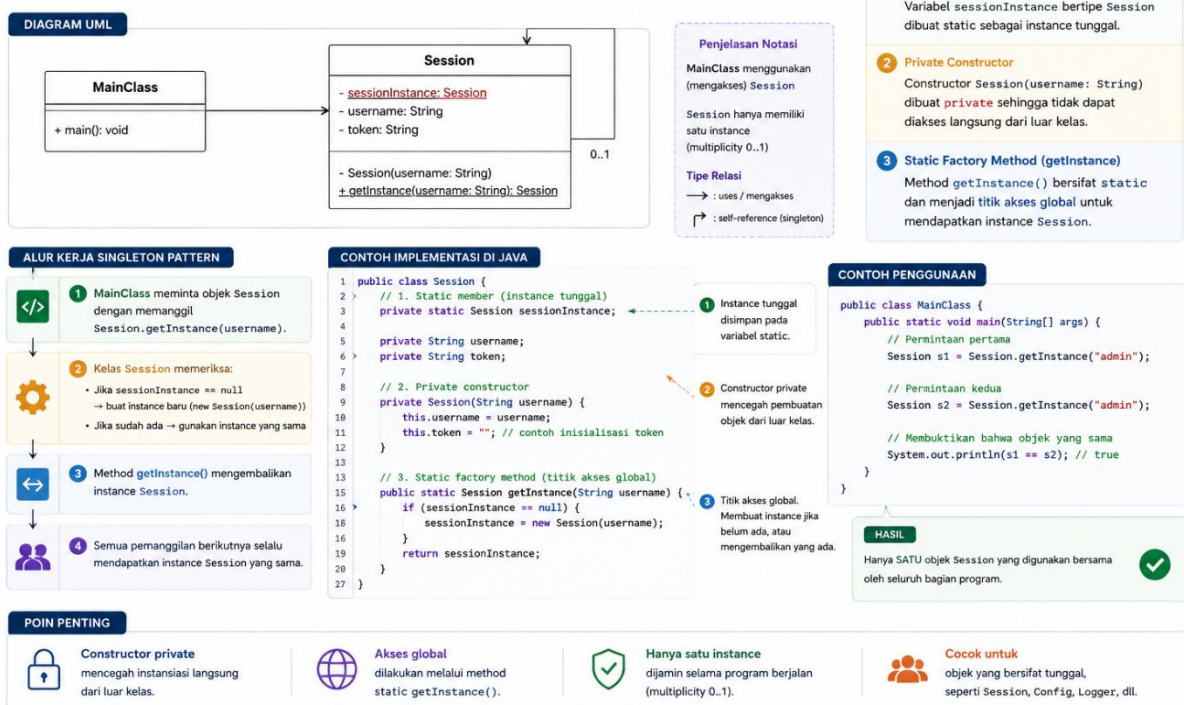
- Menghemat penggunaan memori karena hanya terdapat satu objek.
- Mempermudah pengelolaan resource bersama
- Menyediakan akses global terhadap objek tertentu
- Cocok digunakan untuk konfigurasi atau service yang bersifat *shared*.

Kekurangan Singleton Pattern

- Dapat meningkatkan *coupling* antar kelas apabila digunakan secara berlebihan.
- Menyulitkan proses unit testing karena objek bersifat global
- Pada implementasi multithreading diperlukan mekanisme tambahan agar tetap aman (*thread-safe*)

SINGLETON PATTERN (Contoh: Session)

Diagram berikut menunjukkan implementasi Singleton Pattern pada kelas Session. Hanya ada satu instance Session yang dapat diakses melalui method `getInstance()`.



Berdasarkan contoh diagram di atas, terdapat kelas Session yang menerapkan Singleton Method dengan variabel `sessionInstance` yang dibuat static sebagai instance tunggal, constructor dengan visibilitas *private*, dan akses melalui titik akses method `getInstance()`.

Persiapan Unguided!

- Wajib menyelesaikan UGD 10 Design Pattern 1 beserta bonusnya. Soal unguided akan melanjutkan program yang sudah dibuat pada UGD 10.
- Pelajari kembali materi-materi sebelumnya mengenai konsep Pemrograman Berorientasi Obyek: pewarisan, polimorfisme, relasi (agregasi, komposisi, asosiasi), dan generic class (pelajari juga konsep casting pada generic class).

Referensi

<https://www.geeksforgeeks.org/system-design/software-design-patterns/>

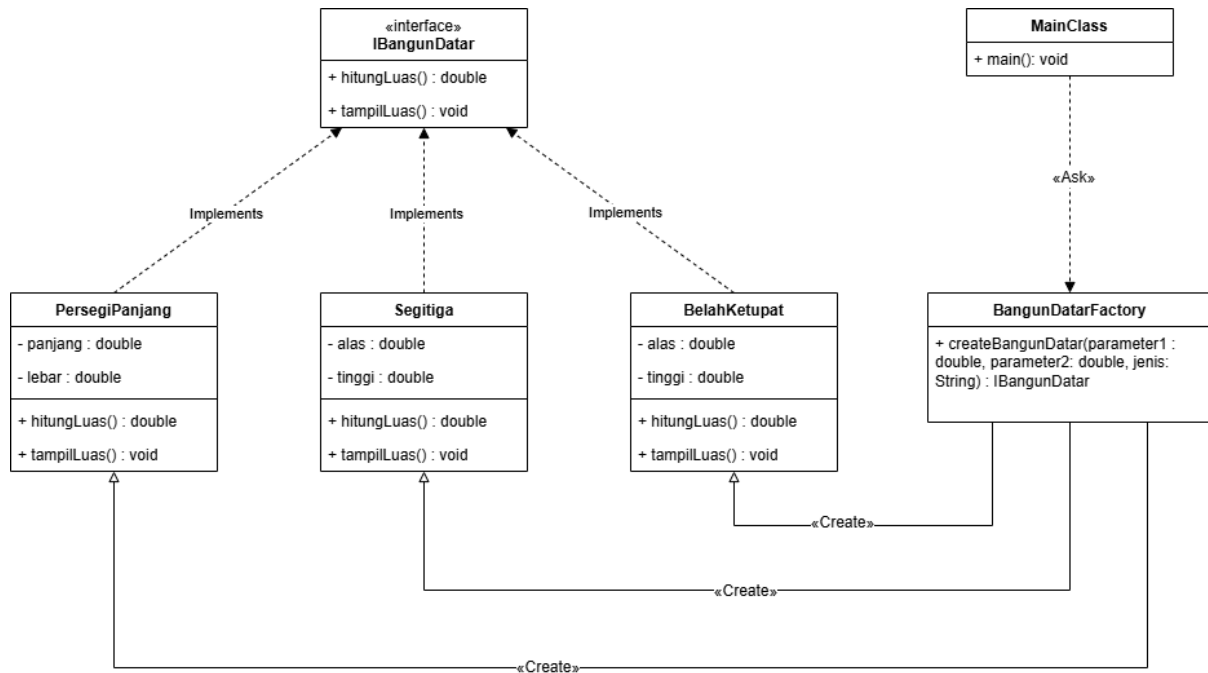
<https://www.geeksforgeeks.org/system-design/factory-method-for-designing-pattern/>

<https://www.geeksforgeeks.org/system-design/singleton-design-pattern/>

Guided Modul 11

Design Pattern 2

Pada Modul 11 -Design Pattern 2, terdapat dua guided yang harus dikerjakan yaitu guided factory method dan singleton method.

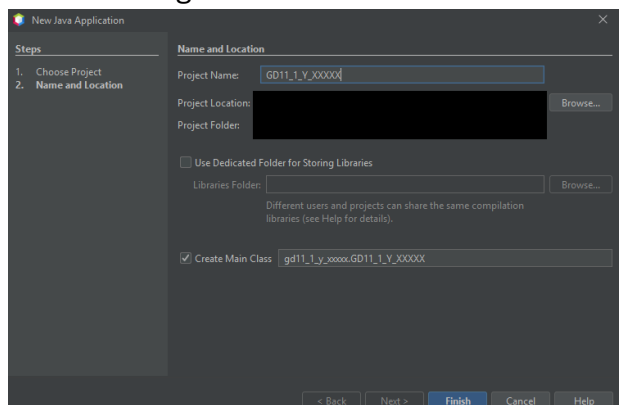


A. Factory Method

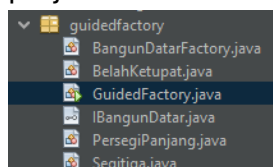
1. Buat project baru dengan nama GD11_1_Y_XXXXX

Y = Kelas

XXXXX = 5 digit NPM



2. Buatlah kelas PersegiPanjang, Segitiga, BelahKetupat, BangunDatarFactory, dan interface IBangunDatar. Penamaan package menyesuaikan dengan nama project kalian masing-masing.



3. IBangunDatar.java

```
/*
    NAMA LENGKAP
    NPM
    KELAS
*/
public interface IBangunDatar {
    public double hitungLuas();
    public void tampilLuas();
}
```

4. PersegiPanjang.java

```
package guidedfactory;

/*
    NAMA LENGKAP
    NPM
    KELAS
*/
public class PersegiPanjang implements IBangunDatar {
    private double panjang;
    private double lebar;

    public PersegiPanjang(double panjang, double lebar) {
        this.panjang = panjang;
        this.lebar = lebar;
    }

    @Override
    public double hitungLuas() {
        return panjang * lebar;
    }

    @Override
    public void tampilLuas() {
        System.out.println("Luas Persegi Panjang = " + hitungLuas() + " cm^2");
    }
}
```

5. Segitiga.java

```

package guidedfactory;

/*
    NAMA LENGKAP
    NPM
    KELAS
*/
public class Segitiga implements IBangunDatar{
    private double alas;
    private double tinggi;

    public Segitiga (double alas, double tinggi) {
        this.alas = alas;
        this.tinggi = tinggi;
    }

    @Override
    public double hitungLuas() {
        return alas * tinggi / 2;
    }

    @Override
    public void tampilLuas() {
        System.out.println("Luas Segitiga = " + hitungLuas() + " cm^2");
    }
}

```

6. BelahKetupat.java

```

package guidedfactory;

/*
    NAMA LENGKAP
    NPM
    KELAS
*/
public class BelahKetupat implements IBangunDatar{
    private double diagonal1;
    private double diagonal2;

    public BelahKetupat(double diagonal1, double diagonal2) {
        this.diagonal1 = diagonal1;
        this.diagonal2 = diagonal2;
    }

    @Override
    public double hitungLuas() {
        return diagonal1 * diagonal2 / 2;
    }

    @Override
    public void tampilLuas() {
        System.out.println("Luas Belah Ketupat = " + hitungLuas() + " cm^2");
    }
}

```

7. BangunDatarFactory.java

```
package guidedfactory;

/*
    NAMA LENGKAP
    NPM
    KELAS
*/

public class BangunDatarFactory {
    public IBangunDatar createBangunDatar(double parameter1, double parameter2, String jenis) {
        if(jenis.equals("Persegi Panjang")) {
            return new PersegiPanjang(parameter1, parameter2);
        } else if(jenis.equals("Segitiga")) {
            return new Segitiga(parameter1, parameter2);
        } else {
            return new BelahKetupat(parameter1, parameter2);
        }
    }
}
```

8. Main Class

```
package guidedfactory;

/*
    NAMA LENGKAP
    NPM
    KELAS
*/

public class GuidedFactory {

    public static void main(String[] args) {
        BangunDatarFactory b = new BangunDatarFactory();

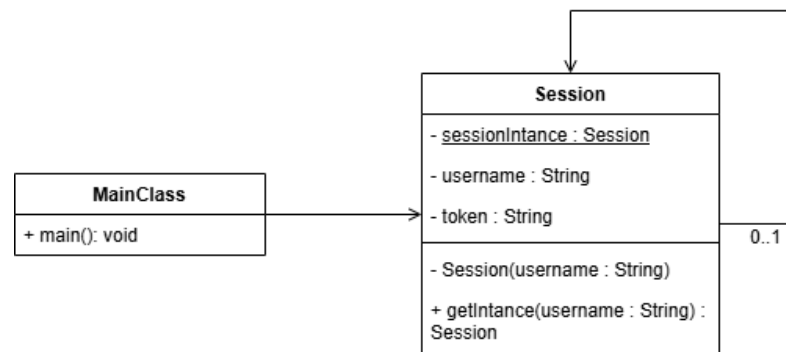
        IBangunDatar b1 = b.createBangunDatar(10.0, 5.0, "Persegi Panjang");
        IBangunDatar b2 = b.createBangunDatar(5.0, 3.0, "Segitiga");
        IBangunDatar b3 = b.createBangunDatar(4.0, 7.0, "Belah Ketupat");

        b1.tampilLuas();
        b2.tampilLuas();
        b3.tampilLuas();
    }
}
```

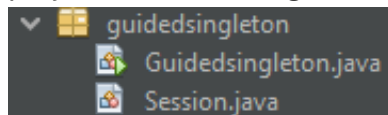
9. Output

```
run:
Luas Persegi Panjang = 50.0 cm^2
Luas Segitiga = 7.5 cm^2
Luas Belah Ketupat = 14.0 cm^2
BUILD SUCCESSFUL (total time: 0 seconds)
```

B. Singleton Method



1. Buat project baru dengan nama GD11_2_Y_XXXXX.
Y = kelas
XXXXX = 5 digit NPM
2. Buatlah kelas Session. Penamaan package menyesuaikan dengan nama project kalian masing-masing.



3. Session.java

```
import java.util.UUID;

/*
    NAMA LENGKAP
    NPM
    KELAS
*/
public class Session {
    private static Session sessionIntance = null;

    private String username;
    private String token;

    private Session(String username) {
        this.username = username;
        this.token = UUID.randomUUID().toString();
    }

    public static Session getInstance(String username) {
        System.out.println("Creating Session ...");
        if(Session.sessionIntance == null) {
            Session.sessionIntance = new Session(username);
            System.out.println("[+] Session Created Successfully");
        } else {
            System.out.println("[!] Session Already Created");
        }
        System.out.println("Username\t: " + Session.sessionIntance.username);
        System.out.println("Token\t\t: " + Session.sessionIntance.token);
        return Session.sessionIntance;
    }
}
```

4. Main Class

```
package guidedsingleton;

/*
    NAMA LENGKAP
    NPM
    KELAS
*/
public class Guidedsingleton {

    public static void main(String[] args) {
        Session session;

        session = Session.getInstance("praktikumPBO");
        session = Session.getInstance("designPattern2");
    }

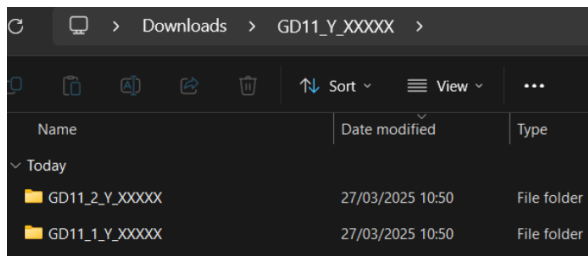
}
```

5. Output

```
run:
Creating Session ...
[+] Session Created Successfully
Username      : praktikumPBO
Token        : 688bb5e5-5429-4ce4-9eb1-8d768a48b964
Creating Session ...
[!] Session Already Created
Username      : praktikumPBO
Token        : 688bb5e5-5429-4ce4-9eb1-8d768a48b964
BUILD SUCCESSFUL (total time: 0 seconds)
```

Pengumpulan Guided

Masukkan folder kedua guided (GD11_1_Y_XXXXX dan GD11_2_Y_XXXXX) ke dalam sebuah folder baru dengan penamaan GD11_Y_XXXXX. Ubah folder ke dalam format .zip (GD11_Y_XXXXX.zip) kemudian kumpulkan melalui uploader yang tersedia pada situs kuliah. Tidak ada toleransi keterlambatan dengan alasan apapun.



Keterangan:

Y = kelas

XXXXX = 5 digit NPM